

**Mathematical Programming and
Operations Research**
Modeling, Algorithms, and Complexity
Examples in Excel and Python
(Work in progress)

Edited by: Robert Hildebrand

Contributors: Robert Hildebrand, Laurent Poirrier, Douglas Bish, Diego Moran

Version Compilation date: August 13, 2025

Preface

This entire book is a working manuscript. The first draft of the book is yet to be completed.

This book is being written and compiled using a number of open source materials. We will strive to properly cite all resources used and give references on where to find these resources. Although the material used in this book comes from a variety of licences, everything used here will be CC-BY-SA 4.0 compatible, and hence, the entire book will fall under a CC-BY-SA 4.0 license.

MAJOR ACKNOWLEDGEMENTS

I would like to acknowledge that substantial parts of this book were borrowed under a CC-BY-SA license. These substantial pieces include:

- "A First Course in Linear Algebra" by Lyryx Learning (based on original text by Ken Kuttler). A majority of their formatting was used along with selected sections that make up the appendix sections on linear algebra. We are extremely grateful to Lyryx for sharing their files with us. They do an amazing job compiling their books and the templates and formatting that we have borrowed here clearly took a lot of work to set up. Thank you for sharing all of this material to make structuring and forming this book much easier! See subsequent page for list of contributors.
- "Foundations of Applied Mathematics" with many contributors. See <https://github.com/Foundations-of-Applied-Mathematics>. Several sections from these notes were used along with some formatting. Some of this content has been edited or rearranged to suit the needs of this book. This content comes with some great references to code and nice formatting to present code within the book. See subsequent page with list of contributors.
- "Linear Inequalities and Linear Programming" by Kevin Cheung. See <https://github.com/dataopt/lineqlpbook>. These notes are posted on GitHub in a ".Rmd" format for nice reading online. This content was converted to $\text{\LaTeX}$ using Pandoc. These notes make up a substantial section of the Linear Programming part of this book.
- Linear Programming notes by Douglas Bish. These notes also make up a substantial section of the Linear Programming part of this book.

I would also like to acknowledge Laurent Porrier and Diego Moran for contributing various notes on linear and integer programming.

I would also like to thank Jamie Fravel for helping to edit this book and for contributing chapters, examples, and code.

Contents

Contents	5
1 Resources and Notation	5
2 Mathematical Programming	9
2.1 Warm-Up Scenario	9
2.2 Why Study Operations Research?	11
2.3 What is Mathematical Programming?	11
2.4 Applications	12
2.5 Types of Optimization problems	13
2.6 Linear Programming (LP)	13
2.7 Mixed-Integer Linear Programming (MILP)	14
2.8 Non-Linear Programming (NLP)	16
2.8.1 Convex Programming	17
2.8.2 Non-Convex Non-linear Programming	17
2.8.3 Machine Learning	18
2.9 Mixed Integer Non-Linear Programming (MINLP)	19
2.9.1 Convex MINLP	20
2.9.2 Non-Convex MINLP	20
2.10 Optimization Under Uncertainty	23
2.10.1 Stochastic Optimization	23
2.10.2 Robust Optimization	23
2.10.3 Applications	23
I Linear Programming	25
3 Modeling: Linear Programming	27
3.1 Introductory LP Example	28
3.2 Modeling and Assumptions in Linear Programming	29
3.2.1 Assumptions	31
3.3 Examples	32
3.3.1 Knapsack Problem	41
3.3.2 Capital Investment	42
3.3.3 Transportation problem	43
3.3.4 Assignment Problem	45
3.4 Exercises	46
3.5 Contributors	48

Introduction

Letter to instructors

This is an introductory book for students to learn optimization theory, tools, and applications. The two main goals are (1) students are able to apply the of optimization in their future work or research (2) students understand the concepts underlying optimization solver and use this knowledge to use solvers effectively.

This book was based on a sequence of course at Virginia Tech in the Industrial and Systems Engineering Department. The courses are *Deterministic Operations Research I* and *Deterministic Operations Reserach II*. The first course focuses on linear programming, while the second course covers integer programming an nonlinear programming. As such, the content in this book is meant to cover 2 or more full courses in optimization.

The book is designed to be read in a linear fashion. That said, many of the chapters are mostly independent and could be rearranged, removed, or shortened as desited. This is an open source textbook, so use it as you like. You are encouraged to share adaptations and improvements so that this work can evolve over time.

Letter to students

This book is designed to be a resource for students interested in learning what optimizaiton is, how it works, and how you can apply it in your future career. The main focus is being able to apply the techniques of optimization to problems using computer technology while understanding (at least at a high level) what the computer is doing and what you can claim about the output from the computer.

Quite importantly, keep in mind that when someone claims to have *optimized* a problem, we want to know what kind of gaurantees they have about how good their solution is. Far too often, the solution that is provided is suboptimal by 10%, 20%, or even more. This can mean spending excess amounts of money, time, or energy that could have been saved. And when problems are at a large scale, this can easily result in millions of dollars in savings.

For this reason, we will learn the perspective of *mathematical programming* (a.k.a. mathematical optimization). The key to this study is that we provide gaurantees on how good a solution is to a given problem. We will also study how difficult a problem is to solve. This will help us know (a) how long it might take to solve it and (b) how good a of a solution we might expect to be able to find in reasonable amount of time.

We will later study *heuristic* methods - these methods typically do not come with gaurantees, but tend to help find quality solutions.

Note: Although there is some computer programming required in this work, this is not a course on programming. Thanks to the fantastic modelling packages available these days, we are able to solve complicated problems with little programming effort. The key skill we will need to learn is *mathematical modeling*: converting words and ideas into numbers and variables in order to communicate problems to a computer so that it can solve a problem for you.

As a main element of this book, we would like to make the process of using code and software as easy as possible. Attached to most examples in the book, there will be links to code that implements and solves the problem using several different tools from Excel and Python. These examples can should make it easy to solve a similar problem with different data, or more generally, can serve as a basis for solving related problems with similar structure.

How to use this book

Skim ahead. We recommend that before you come across a topic in lecture, that you skim the relevant sections ahead of time to get a broad overview of what is to come. This may take only a fraction of the time that it may take for you to read it.

Read the expected outcomes. At the beginning of each section, there will be a list of expected outcomes from that section. Review these outcomes before reading the section to help guide you through what is most relevant for you to take away from the material. This will also provide a brief look into what is to follow in that section.

Read the text. Read carefully the text to understand the problems and techniques. We will try to provide a plethora of examples, therefore, depending on your understanding of a topic, you many need to go carefully over all of the examples.

Explore the resources. Lastly, we recognize that there are many alternative methods of learning given the massive amounts of information and resources online. Thus, at the end of each section, there will be a number of superb resources that available on the internet in different formats. There are other free textbooks, informational websites, and also number of fantastic videos posted to youtube. We encourage to explore the resources to get another perspective on the material or to hear/read it taught from a differnt point of view or in presentation style.

Outline of this book

This book is divided in to 4 Parts:

Part I Linear Programming,

Part II Integer Programming,

Part III Discrete Algorithms,

Part IV Nonlinear Programming.

There are also a number of chapters of background material in the Appendix.

The content of this book is designed to encompass 2-3 full semester courses in an industrial engineering department.

Work in progress

This book is still a work in progress, so please feel free to send feedback, questions, comments, and edits to Robert Hildebrand at open.optimization@gmail.com.

1. Resources and Notation

Here are a list of resources that may be useful as alternative references or additional references.

FREE NOTES AND TEXTBOOKS

- Mathematical Programming with Julia by Richard Lusby & Thomas Stidsen
- Linear Programming by K.J. Mtetwa, David
- A first course in optimization by Jon Lee
- Introduction to Optimizaiton Notes by Komei Fukuda
- Convex Optimization by Bord and Vandenberghe
- LP notes of Michel Goemans from MIT
- Understanding and Using Linear Programming - Matousek and Gärtner [Downloadable from Springer with University account]
- Operations Research Problems Statements and Solutions - Raúl PolerJosefa Mula Manuel Díaz-Madroñero [Downloadable from Springer with University account]

NOTES, BOOKS, AND VIDEOS BY VARIOUS SOLVER GROUPS

- AIMMS Optimization Modeling
- Optimization Modeling with LINGO by Linus Schrage
- The AMPL Book
- Microsoft Excel 2019 Data Analysis and Business Modeling, Sixth Edition, by Wayne Winston - Available to read for free as an e-book through Virginia Tech library at Orielly.com.
- Lesson files for the Winston Book
- Video instructions for solver and an example workbook
- youtube-OR-course

GUROBI LINKS

- Go to <https://github.com/Gurobi> and download the example files.
- Essential ingredients
- Gurobi Linear Programming tutorial
- Gurobi tutorial MILP
- GUROBI - Python 1 - Modeling with GUROBI in Python
- GUROBI - Python II: Advanced Algebraic Modeling with Python and Gurobi
- GUROBI - Python III: Optimization and Heuristics
- Webinar Materials
- GUROBI Tutorials

HOW TO PROVE THINGS

- Hammack - Book of Proof

STATISTICS

- Open Stax - Introductory Statistics

LINEAR ALGEBRA

- Beezer - A first course in linear algebra
- Selinger - Linear Algebra
- Cherney, Denton, Thomas, Waldron - Linear Algebra

REAL ANALYSIS

- Mathematical Analysis I by Elias Zakon

DISCRETE MATHEMATICS, GRAPHS, ALGORITHMS, AND COMBINATORICS

- Levin - Discrete Mathematics - An Open Introduction, 3rd edition
- Github - Discrete Mathematics: an Open Introduction CC BY SA
- Keller, Trotter - Applied Combinatorics (CC-BY-SA 4.0)
- Keller - Github - Applied Combinatorics

MIXED INTEGER NONLINEAR PROGRAMMING

- Mixed-Integer Nonlinear Optimization Pietro Belotti, Christian Kirches, Sven Leyffer, Jeff Linderoth, Jim Luedtke, and Ashutosh Mahajan

PROGRAMMING WITH PYTHON

- A Byte of Python
- Github - Byte of Python (CC-BY-SA)

Also, go to <https://github.com/open-optimization/open-optimization-or-examples> to look at more examples.

Notation

- $\mathbf{1}$ - a vector of all ones (the size of the vector depends on context)
- $\forall$ - for all
- $\exists$ - there exists
- $\in$ - in
- $\therefore$ - therefore
- $\Rightarrow$ - implies
- s.t. - such that (or sometimes "subject to".... from context?)
- $\{0, 1\}$ - the set of numbers 0 and 1
- $\mathbb{Z}$ - the set of integers (e.g. $1, 2, 3, -1, -2, -3, \dots$)
- $\mathbb{Q}$ - the set of rational numbers (numbers that can be written as p/q for $p, q \in \mathbb{Z}$ (e.g. $1, 1/6, 27/2$)
- $\mathbb{R}$ - the set of all real numbers (e.g. $1, 1.5, \pi, e, -11/5$)
- $\setminus$ - setminus, (e.g. $\{0, 1, 2, 3\} \setminus \{0, 3\} = \{1, 2\}$)
- $\cup$ - union (e.g. $\{1, 2\} \cup \{3, 5\} = \{1, 2, 3, 5\}$)

- $\cap$ - intersection (e.g. $\{1, 2, 3, 4\} \cap \{3, 4, 5, 6\} = \{3, 4\}$)
- $\{0, 1\}^4$ - the set of 4 dimensional vectors taking values 0 or 1, (e.g. $[0, 0, 1, 0]$ or $[1, 1, 1, 1]$)
- $\mathbb{Z}^4$ - the set of 4 dimensional vectors taking integer values (e.g., $[1, -5, 17, 3]$ or $[6, 2, -3, -11]$)
- $\mathbb{Q}^4$ - the set of 4 dimensional vectors taking rational values (e.g. $[1.5, 3.4, -2.4, 2]$)
- $\mathbb{R}^4$ - the set of 4 dimensional vectors taking real values (e.g. $[3, \pi, -e, \sqrt{2}]$)
- $\sum_{i=1}^4 i = 1 + 2 + 3 + 4$
- $\sum_{i=1}^4 i^2 = 1^2 + 2^2 + 3^2 + 4^2$
- $\sum_{i=1}^4 x_i = x_1 + x_2 + x_3 + x_4$
- $\square$ - this is a typical Q.E.D. symbol that you put at the end of a proof meaning "I proved it."
- For $x, y \in \mathbb{R}^3$, the following are equivalent (note, in other contexts, these notations can mean different things)

– $x^\top y$ *matrix multiplication*

– $x \cdot y$ *dot product*

– $\langle x, y \rangle$ *inner product*

and evaluate to $\sum_{i=1}^3 x_i y_i = x_1 y_1 + x_2 y_2 + x_3 y_3$.

A sample sentence:

$$\forall x \in \mathbb{Q}^n \exists y \in \mathbb{Z}^n \setminus \{0\}^n \text{ s.t. } x^\top y \in \{0, 1\}$$

"For all non-zero rational vectors x in n -dimensions, there exists a non-zero n -dimensional integer vector y such that the dot product of x with y evaluates to either 0 or 1."

2. Mathematical Programming

Outcomes

- *Consider an introductory problem*
- *Identify reasons for studying operations research*
- *Define "Mathematical Programming"*
- *Learn about different applications of the tools in this book*
- *Explore the different types of optimization models and what types we will see in this book*

2.1 Warm-Up Scenario

Imagine you're starting a new clothing company and decide to sell shirts. You've opened a store in Roanoke and, based on market research, you estimate the demand for the upcoming weekend:

- **Friday:** 5 shirts
- **Saturday:** 10 shirts
- **Sunday:** 7 shirts

Your supplier can deliver shirts in one day, meaning an order placed on Thursday will arrive on Friday, and so on. As a business owner, you need to decide how many shirts to order and when to place each order to ensure you meet the daily demand without overstocking.

*Defining the Problem To tackle this challenge, you'll need to think strategically about two main factors:

- **Ordering Quantities:** How many shirts to order on each day.
- **Inventory Levels:** How many shirts to keep in stock at the end of each day.

You can define these decisions using variables:

- Let the number of shirts you order on a specific day be a decision variable. For example, you could track the number of shirts ordered on Thursday, Friday, and Saturday.
- Define variables to represent the number of shirts you have in stock at the end of each day.

Your goal is to meet the projected demand while managing costs. Costs could include:

- The expense of ordering shirts.

- The cost of holding inventory.
- Penalties for running out of stock.

Refining the Scenario

As your business grows, additional complexities may arise. For instance:

- The cost of ordering shirts might vary by day due to supplier pricing.
- Holding inventory could incur storage costs.
- A budget might limit how much you can spend on orders.
- You might expand your product line to include pants and socks, each with its own demand and costs.

By clearly defining variables and constraints for these decisions, you can systematically approach the problem and develop an optimal strategy. This step-by-step process of defining the problem, incorporating real-world details, and refining the solution is a fundamental aspect of optimization.

Model Development Cycle

This multi-stage example illustrates the iterative cycle of modeling:

1. **Initial Problem:** Determine ordering quantities to meet demand.
2. **Modeling:** Formulate an optimization model based on available information.
3. **Solution:** Solve the model to find the optimal ordering plan.
4. **New Information:** Incorporate additional real-world complexities (varying costs, inventory costs, budget constraints, additional products, fixed ordering costs).
5. **Refinement:** Update the model to reflect new information and solve again.

Further Extensions

Looking ahead, there are several ways to expand this scenario:

- **Expanding to New Stores:** Opening additional stores introduces location-based demand, transportation costs, and coordination of inventory across multiple sites.
- **Hiring New Staff:** Incorporate labor costs, scheduling constraints, and productivity rates into the model.
- **Producing Clothing In-House:** Transitioning to manufacturing involves modeling production processes, including purchasing raw materials, machine capacities, and processing times.
- **Uncertain Demand:** In reality, demand may be uncertain. Advanced models can incorporate stochastic elements or use probabilistic methods to handle randomness.

In this course, we will focus on deterministic models where all data is known and fixed. Building a strong foundation with these models will prepare us to tackle more complex problems involving uncertainty and randomness in future studies.

2.2 Why Study Operations Research?

Operations Research (OR) is a discipline that applies advanced analytical methods to help make better decisions. In a world where resources are limited and organizations face complex challenges, OR provides the tools and methodologies to model, analyze, and solve problems involving the optimal allocation of scarce resources. Studying OR equips individuals with a systematic and quantitative approach to decision-making, which is invaluable in industries such as logistics, finance, healthcare, and manufacturing.

By understanding OR, you gain the ability to construct mathematical models of real-world systems, allowing for the simulation and optimization of different scenarios. This leads to improved efficiency, reduced costs, and enhanced performance. Moreover, the skills developed through studying OR—such as critical thinking, problem-solving, and data analysis—are highly transferable and sought after in today's data-driven economy.

Operations Research also bridges the gap between theory and practice. It transforms abstract mathematical concepts into actionable strategies that can have a tangible impact on organizational success. Whether it's optimizing supply chains, scheduling flights, or managing portfolios, the applications of OR are vast and impactful.

2.3 What is Mathematical Programming?

Mathematical Programming is a core area within Operations Research that focuses on the formulation and solution of optimization problems. It involves creating mathematical models that represent the decision-making process of a system, where the goal is to find the best possible outcome under given constraints. The term "programming" in this context refers to planning or scheduling, not computer programming.

At the heart of Mathematical Programming is the objective function—a mathematical expression that defines the criterion to be optimized, such as minimizing costs or maximizing profits. The decision variables represent the choices available, and the constraints reflect the limitations or requirements of the problem, such as resource capacities or regulatory conditions.

There are various types of mathematical programming, each suited to different kinds of problems:

- **Linear Programming (LP):** Deals with problems where both the objective function and constraints are linear.
- **Integer Programming (IP):** Involves decision variables that must take on integer values.

- **Nonlinear Programming (NLP):** Handles problems with nonlinear objective functions or constraints.
- **Dynamic Programming (DP):** Breaks down problems into simpler subproblems and is particularly useful for multistage decision processes.

Mathematical Programming provides a powerful framework for optimizing complex systems and making informed decisions. It combines elements of mathematics, economics, and computer science to develop solutions that are both efficient and effective.

2.4 Applications

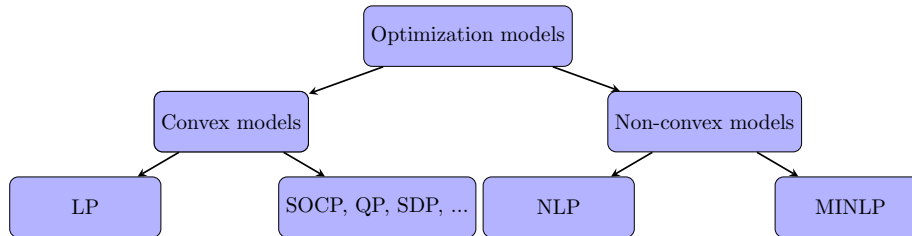
Operations Research and Mathematical Programming have a wide range of applications across various industries and sectors:

- **Transportation and Logistics:** Optimizing routing and scheduling for delivery vehicles, reducing transportation costs, and improving supply chain efficiency.
- **Manufacturing:** Streamlining production processes, managing inventory levels, and scheduling machinery to maximize productivity and minimize waste.
- **Finance:** Portfolio optimization, risk assessment, and capital budgeting to enhance financial performance and manage uncertainties.
- **Healthcare:** Allocating resources such as staff and equipment, scheduling surgeries, and managing patient flow to improve service quality and reduce wait times.
- **Energy Sector:** Optimizing power generation and distribution, planning for renewable energy integration, and managing energy consumption.
- **Telecommunications:** Network design and optimization, bandwidth allocation, and improving the reliability and efficiency of communication systems.
- **Agriculture:** Planning crop rotations, optimizing the use of fertilizers and water, and managing supply chains for agricultural products.
- **Public Sector:** Urban planning, emergency response logistics, and policy analysis to enhance public services and resource utilization.

These applications demonstrate the versatility and impact of OR and Mathematical Programming. By leveraging these tools, organizations can make data-driven decisions that lead to significant improvements in performance, cost savings, and competitive advantage. As global challenges become more complex, the importance of skilled professionals in Operations Research continues to grow.

2.5 Types of Optimization problems

We will state main general problem classes to be associated with in these notes. These are Linear Programming (LP), Mixed-Integer Linear Programming (MILP), Non-Linear Programming (NLP), and Mixed-Integer Non-Linear Programming (MINLP).



Along with each problem class, we will associate a complexity class for the general version of the problem. Complexity classes are discussed in a later portion of the book. Although we will often state that input data for a problem comes from $\mathbb{R}$ (as a real number), when we discuss complexity of such a problem, we actually mean that the data is rational, i.e., from $\mathbb{Q}$ (rational numbers that are encoded in the computer typically), and is given in binary encoding.

CAUTION!

In the following subsections, we will use advanced notation that may be unfamiliar at this point. These mathematical notations will be developed throughout the book.

Furthermore, don't be too concerned about the specific definitions of the complexity classes at this point. For now, you can read that *Polynomial time (P)* means it should be relatively easy to solve, while the other classes might be much more difficult to solve.

2.6 Linear Programming (LP)

Some linear programming background, theory, and examples will be provided in ??.

Linear Programming (LP):

Polynomial time (P)

Given a matrix $A \in \mathbb{R}^{m \times n}$, vector $b \in \mathbb{R}^m$ and vector $c \in \mathbb{R}^n$, the *linear programming* problem is

$$\begin{aligned}
 \max \quad & c^\top x \\
 \text{s.t.} \quad & Ax \leq b \\
 & x \geq 0
 \end{aligned} \tag{2.1}$$

Linear programming can come in several forms, whether we are maximizing or minimizing, or if the constraints are $\leq$, $=$ or $\geq$. One form commonly used is *Standard Form* given as

Linear Programming (LP) Standard Form:

Polynomial time (P)

Given a matrix $A \in \mathbb{R}^{m \times n}$, vector $b \in \mathbb{R}^m$ and vector $c \in \mathbb{R}^n$, the *linear programming* problem in *standard form* is

$$\begin{aligned} \max \quad & c^\top x \\ \text{s.t.} \quad & Ax = b \\ & x \geq 0 \end{aligned} \tag{2.2}$$

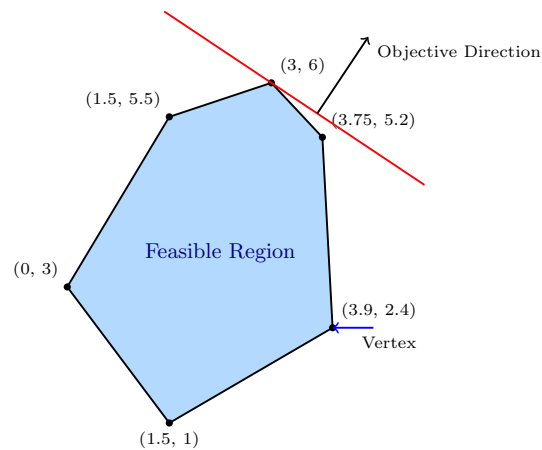


Figure 2.1: Linear programming constraints and objective.

2.7 Mixed-Integer Linear Programming (MILP)

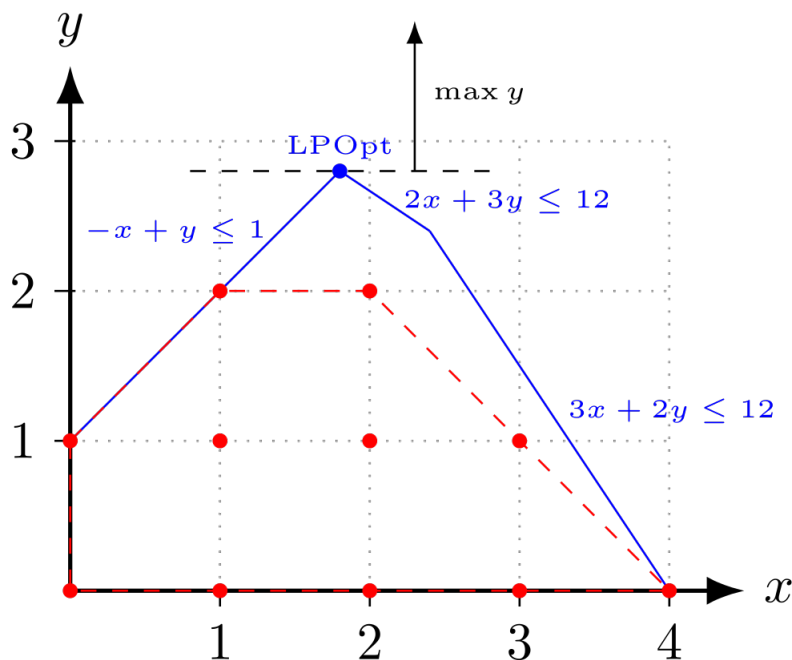
Mixed-integer linear programming will be the focus of Sections ??, ??, ??, and ??. Recall that the notation $\mathbb{Z}$ means the set of integers and the set $\mathbb{R}$ means the set of real numbers. The first problem of interest here is a *binary integer program* (BIP) where all n variables are binary (either 0 or 1).

Binary Integer programming (BIP):*NP-Complete*

Given a matrix $A \in \mathbb{R}^{m \times n}$, vector $b \in \mathbb{R}^m$ and vector $c \in \mathbb{R}^n$, the *binary integer programming* problem is

$$\begin{aligned} \max \quad & c^\top x \\ \text{s.t.} \quad & Ax \leq b \\ & x \in \{0, 1\}^n \end{aligned} \tag{2.1}$$

A slightly more general class is the class of *Integer Linear Programs* (ILP). Often this is referred to as *Integer Program* (IP), although this term could leave open the possibility of non-linear parts.



© wiki/File/integer-programming.png¹

Figure 2.2: Comparing the LP relaxation to the IP solutions.

Figure 2.2

Integer Linear Programming (ILP):*NP-Complete*

¹wiki/File/integer-programming.png, from wiki/File/integer-programming.png. wiki/File/integer-programming.png, wiki/File/integer-programming.png.

Given a matrix $A \in \mathbb{R}^{m \times n}$, vector $b \in \mathbb{R}^m$ and vector $c \in \mathbb{R}^n$, the *integer linear programming* problem is

$$\begin{aligned} \max \quad & c^\top x \\ \text{s.t.} \quad & Ax \leq b \\ & x \in \mathbb{Z}^n \end{aligned} \tag{2.2}$$

An even more general class is *Mixed-Integer Linear Programming (MILP)*. This is where we have n integer variables $x_1, \dots, x_n \in \mathbb{Z}$ and d continuous variables $x_{n+1}, \dots, x_{n+d} \in \mathbb{R}$. Succinctly, we can write this as $x \in \mathbb{Z}^n \times \mathbb{R}^d$, where $\times$ stands for the *cross-product* between two spaces.

Below, the matrix A now has $n + d$ columns, that is, $A \in \mathbb{R}^{m \times (n+d)}$. Also note that we have not explicitly enforced non-negativity on the variables. If there are non-negativity restrictions, this can be assumed to be a part of the inequality description $Ax \leq b$.

Mixed-Integer Linear Programming (MILP):

NP-Complete

Given a matrix $A \in \mathbb{R}^{m \times (n+d)}$, vector $b \in \mathbb{R}^m$ and vector $c \in \mathbb{R}^{n+d}$, the *mixed-integer linear programming* problem is

$$\begin{aligned} \max \quad & c^\top x \\ \text{s.t.} \quad & Ax \leq b \\ & x \in \mathbb{Z}^n \times \mathbb{R}^d \end{aligned} \tag{2.3}$$

2.8 Non-Linear Programming (NLP)

NLP:

NP-Hard

Given a function $f(x): \mathbb{R}^d \rightarrow \mathbb{R}$ and other functions $f_i(x): \mathbb{R}^d \rightarrow \mathbb{R}$ for $i = 1, \dots, m$, the *nonlinear programming* problem is

$$\begin{aligned} \min \quad & f(x) \\ \text{s.t.} \quad & f_i(x) \leq 0 \quad \text{for } i = 1, \dots, m \\ & x \in \mathbb{R}^d \end{aligned} \tag{2.1}$$

Nonlinear programming can be separated into convex programming and non-convex programming. These two are very different beasts and it is important to distinguish between the two.

2.8.1. Convex Programming

Here the functions are all **convex**!

Convex Programming:

Polynomial time (P) (typically)

Given a convex function $f(x): \mathbb{R}^d \rightarrow \mathbb{R}$ and convex functions $f_i(x): \mathbb{R}^d \rightarrow \mathbb{R}$ for $i = 1, \dots, m$, the *convex programming* problem is

$$\begin{aligned} \min \quad & f(x) \\ \text{s.t.} \quad & f_i(x) \leq 0 \quad \text{for } i = 1, \dots, m \\ & x \in \mathbb{R}^d \end{aligned} \tag{2.2}$$

Observe that convex programming is a generalization of linear programming. This can be seen by letting $f(x) = c^\top x$ and $f_i(x) = A_i x - b_i$.

2.8.2. Non-Convex Non-linear Programming

When the function f or functions f_i are non-convex, this becomes a non-convex nonlinear programming problem. There are a few complexity issues with this.

IP AS NLP As seen above, quadratic constraints can be used to create a feasible region with discrete solutions. For example

$$x(1 - x) = 0$$

has exactly two solutions: $x = 0, x = 1$. Thus, quadratic constraints can be used to model binary constraints.

Binary Integer programming (BIP) as a NLP:

NP-Hard

Given a matrix $A \in \mathbb{R}^{m \times n}$, vector $b \in \mathbb{R}^m$ and vector $c \in \mathbb{R}^n$, the *binary integer programming* problem

is

$$\begin{aligned}
 \max \quad & c^\top x \\
 \text{s.t.} \quad & Ax \leq b \\
 & \cancel{x \in \{0,1\}^n} \\
 & x_i(1-x_i) = 0 \quad \text{for } i = 1, \dots, n
 \end{aligned} \tag{2.3}$$

Alternatively, consider the transformation where $x_i \in \{-1, 1\}$.

$$\begin{aligned}
 \min \quad & c^\top x \\
 \text{s.t.} \quad & Ax \leq b \\
 & x \in \{-1, 1\}^n
 \end{aligned}$$

This can be reformulated with a single nonconvex constraint as

$$\begin{aligned}
 \min \quad & c^\top x \\
 \text{s.t.} \quad & Ax \leq b \\
 & -1 \leq x_j \leq 1, \quad 1 \leq j \leq n, \\
 & \|x\|^2 \geq n.
 \end{aligned}$$

2.8.3. Machine Learning

Machine learning problems are often cast as continuous optimization problems, which involve adjusting parameters to minimize or maximize a particular objective. Frequently they are convex optimization problems, but many turn out to be nonconvex. Here are two examples of how these problems arise at a glance. We will see examples in greater detail later in the book.

Loss Function Minimization

In supervised learning, this objective is typically a loss function L that quantifies the discrepancy between the predictions of a model and the true data labels. The aim is to adjust the parameters θ of the model to minimize this loss. Mathematically, this can be represented as:

$$\min_{\theta} L(\theta) = \min_{\theta} \frac{1}{N} \sum_{i=1}^N l(y_i, f(x_i; \theta)) \tag{2.4}$$

where N is the number of data points, l is a per-data-point loss (e.g., squared error for regression or cross-entropy for classification), y_i is the true label for the i -th data point, and $f(x_i; \theta)$ is the model's prediction for the i -th data point with parameters θ .

Clustering Formulation

Clustering, on the other hand, seeks to group or partition data points such that data points in the same group are more similar to each other than those in other groups. One popular method is the k-means clustering algorithm. The objective of k-means is to partition the data into k clusters by minimizing the within-cluster sum of squares (WCSS). The mathematical formulation can be given as:

$$\min_{\mathbf{c}_1, \dots, \mathbf{c}_k} \sum_{j=1}^k \sum_{x_i \in C_j} \|x_i - \mathbf{c}_j\|^2 \quad (2.5)$$

where C_j represents the j -th cluster and $\mathbf{c}_j$ is the centroid of that cluster.

This encapsulation presents a glimpse into how ML problems are framed mathematically. In practice, numerous algorithms, constraints, and regularizations add complexity to these basic formulations.

2.9 Mixed Integer Non-Linear Programming (MINLP)

MINLP:

NP-Hard

Mixed Integer Nonlinear Programming (MINLP) combines elements of integer programming and non-linear programming. In MINLP, some or all of the decision variables are constrained to be integers, and the objective function or constraints are nonlinear. A general MINLP problem can be formulated as:

$$\begin{aligned} \min \quad & f(x, y) \\ \text{s.t.} \quad & g_i(x, y) \leq 0 \quad \text{for } i = 1, \dots, m \\ & h_j(x, y) = 0 \quad \text{for } j = 1, \dots, p \\ & x \in \mathbb{R}^n, y \in \mathbb{Z}^k \end{aligned} \quad (2.1)$$

where f is the objective function, g_i and h_j are constraint functions, x is a vector of continuous variables, and y is a vector of integer variables.

MINLP is particularly challenging due to the nonlinearity in the objective and/or constraints and the discrete nature of some decision variables. It finds applications in various fields such as industry for process optimization, computational geometry, and machine learning for hyperparameter tuning.

2.9.1. Convex MINLP

In the convex case, both the objective function and the constraints are convex functions. This subclass is easier to solve compared to its non-convex counterpart.

Convex MINLP:

NP-Hard, but *Polynomial time (P)* in fixed dimension (typically)

For a convex MINLP, the problem is defined as:

$$\begin{aligned} \min \quad & f(x,y) \quad (\text{convex}) \\ \text{s.t.} \quad & g_i(x,y) \leq 0 \quad (\text{convex constraints}) \\ & x \in \mathbb{R}^n, y \in \mathbb{Z}^k \end{aligned} \tag{2.2}$$

Convex MINLPs, while still challenging, are typically more tractable due to the properties of convexity, which allow for more efficient solution methods.

2.9.2. Non-Convex MINLP

Non-convex MINLPs are significantly harder due to the presence of non-convex functions, which can lead to multiple local minima.

Non-Convex MINLP:

NP-Hard (in fact, undecidable)

The non-convex MINLP problem is formulated as:

$$\begin{aligned} \min \quad & f(x,y) \quad (\text{non-convex}) \\ \text{s.t.} \quad & g_i(x,y) \leq 0 \quad (\text{possibly non-convex constraints}) \\ & x \in \mathbb{R}^n, y \in \mathbb{Z}^k \end{aligned} \tag{2.3}$$

Non-convex MINLPs pose significant computational challenges due to the possibility of multiple local optima and the inherent complexity of integer constraints. These problems are common in real-world applications where decisions are discrete, and the system behavior is non-linear and complex.

Complexity and Applications

MINLP problems are known for their computational complexity, primarily due to the combination of non-linearity and integrality. The non-convex variants, in particular, are *NP-Hard*, making them some of the most challenging problems in optimization.

In practice, MINLP models find extensive applications across various domains. In industry, they are used for complex decision-making processes like supply chain optimization and production planning. In computational geometry, MINLP techniques help in solving problems like optimal packing or layout design. Furthermore, in the realm of machine learning, MINLPs are crucial for tasks like feature selection and hyperparameter optimization where discrete choices and nonlinear relationships are involved.

The versatility of MINLP models, combined with their inherent complexity, makes them a fascinating and essential area of study in the field of optimization.

Category	Software (License)
Linear programming (free)	CDD, CLP, GLPK, LPSOLVE, OSQP, SCIP
Mixed Integer Linear programming (free)	CBC, GLPK, LPSOLVE, SCIP
Linear programming (commercial)	COPT (free for academia), CPLEX (free for academia), GUROBI (free for academia), KNITRO, LINPROG, MOSEK (free for academia), XPRESS (free for academia)
Mixed Integer Linear programming (commercial)	COPT (free for academia), CPLEX (free for academia), GUROBI (free for academia), INTLINPROG, KNITRO, MOSEK (free for academia), XPRESS (free for academia)
Quadratic programming (free)	OSQP, CLP, DAQP (mixed-binary), OOQP, QPC, QPOASES, QUADPROGGBB (nonconvex QP)
Quadratic programming (commercial)	COPT (free for academia), CPLEX (free for academia), GUROBI (free for academia), KNITRO, MOSEK (free for academia), QUADPROG, XPRESS (free for academia)
Mixed Integer Quadratic programming (commercial)	COPT (free for academia), CPLEX (free for academia), GUROBI (free for academia), KNITRO, MOSEK (free for academia), XPRESS (free for academia)
Second-order cone programming (free)	ECOS, SCS, SDPT3, SEDUMI
Second-order cone programming (commercial)	COPT (free for academia), CPLEX (free for academia), CONEPROG, GUROBI (free for academia), MOSEK (free for academia), XPRESS (free for academia)
Mixed Integer Second-order cone programming (commercial)	COPT (free for academia), CPLEX (free for academia), GUROBI (free for academia), MOSEK (free for academia), XPRESS (free for academia)
Semidefinite programming (free)	CSDP, DSDP, LOGDETTPPA, PENLAB, SCS, SDPA, SDPLR, SDPT3, SDPNAL, SEDUMI
Semidefinite programming (commercial)	COPT (free for academia), LMILab (not recommended), MOSEK (free for academia), PENBMI, PENSDP (free for academia)
General nonlinear programming and other solvers	BARON, FILTERSD, FMINCON, GPPSOY, IPOPT, KNITRO, LMIRank, MPT, NOMAD, PENLAB, SNOPT, SPARSEPOP

Table 2.1: Optimization Software and their Licenses

2.10 Optimization Under Uncertainty

Optimization problems often involve uncertainty in their parameters, whether due to measurement errors, variability in input data, or unpredictable future events. Addressing uncertainty is crucial for developing solutions that are effective in real-world settings. Two primary approaches to optimization under uncertainty are stochastic optimization and robust optimization.

2.10.1. Stochastic Optimization

Stochastic optimization deals with problems where some parameters are uncertain but can be described probabilistically. In this framework, the optimization process incorporates the expected values, variances, or other statistical characteristics of uncertain parameters. Solutions are often evaluated based on their performance over a distribution of possible scenarios.

For example, in supply chain management, uncertain demand can be modeled using probability distributions. A company may aim to minimize costs while ensuring a high service level under varying demand conditions. Similarly, in finance, portfolio optimization often accounts for uncertain future returns by maximizing expected returns while managing risk.

2.10.2. Robust Optimization

Robust optimization, on the other hand, focuses on creating solutions that perform well across the worst-case realizations of uncertainty. Rather than assuming a specific probability distribution, robust optimization works with uncertainty sets, which define the range of possible values that uncertain parameters can take. The goal is to find solutions that are feasible and effective for all possible scenarios within the uncertainty set.

Applications of robust optimization include power grid operations, where uncertainties in demand and renewable energy generation need to be managed, and scheduling problems, such as airline crew scheduling, where disruptions must be minimized under varying operational conditions.

2.10.3. Applications

Optimization under uncertainty has a wide array of applications across industries:

- **Healthcare:** Designing vaccination strategies to minimize the spread of diseases while accounting for uncertain infection rates and patient behaviors.
- **Transportation:** Managing traffic flow or public transit schedules under uncertain demand and travel times.
- **Energy:** Planning renewable energy investments and operations under uncertain weather conditions and energy prices.

- **Manufacturing:** Ensuring production schedules are resilient to supply chain disruptions and variability in raw material quality.

Addressing the complexity of uncertain data typically makes these problems much harder than their deterministic counterparts. While these methods are powerful and applicable to a wide range of real-world scenarios, we will not focus on optimization under uncertainty in this textbook.

Part I

Linear Programming

3. Modeling: Linear Programming

Outcomes

1. Define what a linear program is
2. Understand how to model a linear program
3. View many examples and get a sense of what types of problems can be modeled as linear programs.

Resources

- Youtube! - Modeling with Linear Programming - Watch 0:00 - 4:19
- Youtube! - Watch 0:00 - 1:43

Linear Programming, also known as Linear Optimization, is the starting point for most forms of optimization. It is the problem of optimization a linear function over linear constraints.

In this chapter, we will define what this means, how to setup a linear program, and discuss many examples. Examples will be connected with code in Excel and Python (using with PuLP or Gurobipy modeling tools) so that you can easily start solving optimization problems. Tutorials on these tools will come in later chapters.

As a warm up, consider the following word problem.

Question: Find the age of Justin and Brad, given that the sum of their ages is equal to 76 and after 7 years, the age of Brad will be twice the age of Justin.

Solution: Let x and y denote the age of Justin and Brad, respectively. Then

$$x + y = 76 \tag{1}$$

$$2(x + 7) = y + 7. \tag{2}$$

On solving the foregoing equations we get

$$x = 23 \quad \text{and} \quad y = 53.$$

We will need to think about assigning variables to decisions and instead of equations guiding these variables, we may have inequalities that bound these variables in different ways. Thus, there can be many possible solutions.

3.1 Introductory LP Example

We begin with a simple example.

Example 3.1: Toy Maker

Excel PuLP Gurobipy

Consider the problem of a toy company that produces toy planes and toy boats. The toy company can sell its planes for \$10 and its boats for \$8 dollars. It costs \$3 in raw materials to make a plane and \$2 in raw materials to make a boat. A plane requires 3 hours to make and 1 hour to finish while a boat requires 1 hour to make and 2 hours to finish. The toy company knows it will not sell anymore than 35 planes per week. Further, given the number of workers, the company cannot spend anymore than 160 hours per week finishing toys and 120 hours per week making toys. The company wishes to maximize the profit it makes by choosing how much of each toy to produce.

We can represent the profit maximization problem of the company as a linear programming problem. Let x_1 be the number of planes the company will produce and let x_2 be the number of boats the company will produce. The profit for each plane is $\$10 - \$3 = \$7$ per plane and the profit for each boat is $\$8 - \$2 = \$6$ per boat. Thus the total profit the company will make is:

$$z(x_1, x_2) = 7x_1 + 6x_2 \quad (3.1)$$

The company can spend no more than 120 hours per week making toys and since a plane takes 3 hours to make and a boat takes 1 hour to make we have:

$$3x_1 + x_2 \leq 120 \quad (3.2)$$

Likewise, the company can spend no more than 160 hours per week finishing toys and since it takes 1 hour to finish a plane and 2 hour to finish a boat we have:

$$x_1 + 2x_2 \leq 160 \quad (3.3)$$

Finally, we know that $x_1 \leq 35$, since the company will make no more than 35 planes per week. Thus the complete linear programming problem is given as:

$$\begin{array}{ll} \max & z(x_1, x_2) = 7x_1 + 6x_2 \\ \text{s.t.} & 3x_1 + x_2 \leq 120 \\ & x_1 + 2x_2 \leq 160 \\ & x_1 \leq 35 \\ & x_1 \geq 0 \\ & x_2 \geq 0 \end{array} \quad (3.4)$$

Exercise 3.1: Chemical Manufacturing

A chemical manufacturer produces three chemicals: A, B and C. These chemical are produced by two processes: 1 and 2.

- Running process 1 for 1 hour costs \$4 and yields 3 units of chemical A, 1 unit of chemical B and 1 unit of chemical C.
- Running process 2 for 1 hour costs \$1 and produces 1 units of chemical A, and 1 unit of chemical B (but none of Chemical C).

To meet customer demand,

- at least 10 units of chemical A,
- at least 5 units of chemical B and
- at least 3 units of chemical C

must be produced daily.

Assume that the chemical manufacturer wants to minimize the cost of production.

Develop a linear programming problem describing the constraints and objectives of the chemical manufacturer.

[Hint: Let x_1 be the amount of time Process 1 is executed and let x_2 be amount of time Process 2 is executed. Use the coefficients above to express the cost of running Process 1 for x_1 time and Process 2 for x_2 time. Do the same to compute the amount of chemicals A, B, and C that are produced.]

3.2 Modeling and Assumptions in Linear Programming

Outcomes

1. Address crucial assumptions when chosing to model a problem with linear programming.

A Generic Linear Program (LP)

Linear programming (LP) is a mathematical optimization framework widely used in decision-making and resource allocation. In its most basic form, an LP seeks to optimize (maximize or minimize) a linear objective function subject to a set of linear constraints. These constraints typically represent limitations or requirements in real-world scenarios, such as budgets, capacities, or resource availability.

Decision Variables:

The decision variables represent the quantities we want to determine. For example:

- x_i : Continuous variables ($x_i \in \mathbb{R}$, i.e., a real number), $\forall i = 1, \dots, n$.

These variables could represent amounts of products to produce, resources to allocate, or investments to make. We can use any letters to represent these variables. Typically we use x_i , but frequently will use y_i or other letters depending on the context and what makes reading the problem most convenient.

Parameters (Known Input Parameters):

Parameters are the given data in the problem, which are assumed to be known in advance:

- c_i : Cost coefficients for each decision variable, $\forall i = 1, \dots, n$. These values represent the contribution of each variable to the objective function.
- a_{ij} : Constraint coefficients, $\forall i = 1, \dots, n$ and $j = 1, \dots, m$. These define the relationships between decision variables and the constraints.
- b_j : Right-hand side values of the constraints, $\forall j = 1, \dots, m$. These represent limits or requirements for each constraint.

The general form of a linear program can be written as follows:

$$\begin{aligned}
 \max \quad & z = c_1x_1 + \cdots + c_nx_n \\
 \text{s.t.} \quad & a_{11}x_1 + \cdots + a_{1n}x_n \leq b_1 \\
 & \vdots \\
 & a_{m1}x_1 + \cdots + a_{mn}x_n \leq b_m \\
 & x_i \geq 0 \quad \forall i = 1, \dots, n.
 \end{aligned} \tag{3.1}$$

Features of a Linear Program

- **Objective Function:** The linear function $z = c_1x_1 + \cdots + c_nx_n$ represents the goal to optimize. This could involve maximizing profit, minimizing cost, or achieving some other measurable outcome.
- **Constraints:** The system of inequalities (or equalities) restricts the values of the decision variables. Each constraint reflects a real-world limitation, such as resource availability or capacity limits.
- **Feasibility:** The set of all possible solutions that satisfy the constraints is called the feasible region. The optimal solution must lie within this region.
- **Linear Relationships:** Both the objective function and constraints are linear, meaning the relationships between variables are additive and proportional.
- **Sets:** For specific optimization problem, it can be useful to define sets of variables or parameters to help write down the problem in a generic formulation.

Example of a Linear Program

To illustrate, consider a specific example with three decision variables and four constraints. This includes a mix of constraint types ($\leq$, $\geq$, and $=$), showcasing the versatility of linear programming:

Decision Variables:

x_i : Continuous variables ($x_i \in \mathbb{R}$, i.e., a real number), $\forall i = 1, \dots, 3$.

Parameters (Known Input Parameters):

- c_i : Cost coefficients, $\forall i = 1, \dots, 3$.
- a_{ij} : Constraint coefficients, $\forall i = 1, \dots, 3$ and $j = 1, \dots, 4$.
- b_j : Right-hand side values for each constraint, $\forall j = 1, \dots, 4$.

The linear program can be written as:

$$\text{Minimize } z = c_1x_1 + c_2x_2 + c_3x_3 \quad (3.2)$$

$$\text{Subject to } a_{11}x_1 + a_{12}x_2 + a_{13}x_3 \geq b_1 \quad (3.3)$$

$$a_{21}x_1 + a_{22}x_2 + a_{23}x_3 \leq b_2 \quad (3.4)$$

$$a_{31}x_1 + a_{32}x_2 + a_{33}x_3 = b_3 \quad (3.5)$$

$$a_{41}x_1 + a_{42}x_2 + a_{43}x_3 \geq b_4 \quad (3.6)$$

$$x_1 \geq 0, x_2 \leq 0, x_3 \text{ unrestricted.} \quad (3.7)$$

A key feature of linear programming is the use of linear functions, defined as follows:

Definition 3.2: Linear Function

A function $z : \mathbb{R}^n \rightarrow \mathbb{R}$ is linear if there are constants $c_1, \dots, c_n \in \mathbb{R}$ such that:

$$z(x_1, \dots, x_n) = c_1x_1 + \dots + c_nx_n. \quad (3.8)$$

Linear functions are characterized by their simplicity and additivity, making them well-suited for optimization problems where proportionality is assumed. This simplicity allows linear programs to be solved efficiently, even for large-scale problems.

3.2.1. Assumptions

Inspecting Example 3.1 (or the more general problem in Equation (3.1)) we can see there are several assumptions that must be satisfied when using a linear programming model. We enumerate these below:

Proportionality Assumption A problem can be phrased as a linear program only if the contribution to the objective function *and* the left-hand-side of each constraint by each decision variable ($x_1, \dots, x_n$) is proportional to the value of the decision variable.

Additivity Assumption A problem can be phrased as a linear programming problem only if the contribution to the objective function *and* the left-hand-side of each constraint by any decision variable x_i ($i = 1, \dots, n$) is completely independent of any other decision variable x_j ($j \neq i$) and additive.

Divisibility Assumption A problem can be phrased as a linear programming problem only if the quantities represented by each decision variable are infinitely divisible (i.e., fractional answers make sense).

Certainty Assumption A problem can be phrased as a linear programming problem only if the coefficients in the objective function and constraints are known with certainty.

The first two assumptions simply assert (in English) that both the objective function and functions on the left-hand-side of the (in)equalities in the constraints are linear functions of the variables $x_1, \dots, x_n$.

The third assumption asserts that a valid optimal answer could contain fractional values for decision variables. It's important to understand how this assumption comes into play—even in the toy making example. Many quantities can be divided into non-integer values (ounces, pounds etc.) but many other quantities cannot be divided. For instance, can we really expect that it's reasonable to make $\frac{1}{2}$ a plane in the toy making example? When values must be constrained to true integer values, the linear programming problem is called an *integer programming problem*. There is a *vast* literature dealing with these problems. For many problems, particularly when the values of the decision variables may become large, a fractional optimal answer could be obtained and then rounded to the nearest integer to obtain a reasonable answer. For example, if our toy problem were re-written so that the optimal answer was to make 1045.3 planes, then we could round down to 1045.

The final assumption asserts that the coefficients (e.g., profit per plane or boat) is known with absolute certainty. In traditional linear programming, there is no lack of knowledge about the make up of the objective function, the coefficients in the left-hand-side of the constraints or the bounds on the right-hand-sides of the constraints. There is a literature on *stochastic programming* that relaxes some of these assumptions, but this too is outside the scope of the course.

Exercise 3.3

In a short sentence or two, discuss whether the problem given in Example 3.1 meets all of the assumptions of a scenario that can be modeled by a linear programming problem. Do the same for Exercise 3.1.

[Hint: Can you make $\frac{2}{3}$ of a toy? Can you run a process for $\frac{1}{3}$ of an hour?]

3.3 Examples

Outcomes

1. Identify decision variables and parameters
2. Learn how to format a linear optimization problem.
3. Identify and understand common classes of linear optimization problems.

We will begin with a few examples, and then discuss specific problem types that occur often.

Example 3.2: Production Planning

Excel PuLP Gurobipy

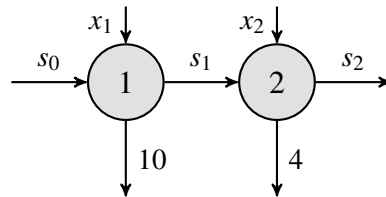
A manufacturing firm seeks to optimize its production schedule over a two-day planning horizon to meet daily demand while minimizing costs. The firm starts with an initial inventory of s_0 units. The costs associated with production and inventory are as follows:

- **Production costs:** \$10 per unit on Day 1, and \$16 per unit on Day 2.
- **Inventory cost:** \$5 per unit for carrying excess inventory from one day to the next.

Production

Inventory

Demand



The goal is to determine the number of units to produce each day and the inventory levels to minimize total costs while satisfying the demand for each day.

Solution. Sets:

- Days of production = $\{1, 2\}$.

Parameters:

- c_{p1} : Per unit production cost on Day 1 (\$10).
- c_{p2} : Per unit production cost on Day 2 (\$16).
- c_{inv} : Per unit inventory cost (\$5).
- d_1 : Demand on Day 1 (10 units).
- d_2 : Demand on Day 2 (4 units).
- s_0 : Inventory at the start of Day 1.

Decision variables:

- x_1 : Number of units produced on Day 1.
- x_2 : Number of units produced on Day 2.
- s_1 : Inventory at the end of Day 1.
- s_2 : Inventory at the end of Day 2.

Objective and Constraints:

$$\begin{array}{ll}
 \min & z = 10x_1 + 16x_2 + 5s_1 + 5s_2 & \text{(Total cost)} \\
 \text{s.t.} & s_0 + x_1 = 10 + s_1 & \text{(Day 1 balance)} \\
 & s_1 + x_2 = 4 + s_2 & \text{(Day 2 balance)} \\
 & x_1, x_2, s_0, s_1, s_2 \geq 0 & \text{(Non-negativity constraints).}
 \end{array}$$

**Definition 3.4: Optimization model components**

When defining an optimization model, it is essential to identify and clearly articulate the following five components:

- **Sets:** What lists of data do you need to write down? These could include indices, categories, or groups relevant to the problem.
- **Parameters (Data):** What is the actual data for the problem? These are the fixed numerical values such as costs, capacities, or resource availabilities.
- **Decision Variables:** What decisions need to be made to provide a solution? These variables represent the quantities that can be controlled or adjusted, such as production levels or resource allocations.
- **Objective Function:** The expression that we want to maximize or minimize. This is the goal of the optimization, such as minimizing cost or maximizing profit.
- **Constraints:** Certain rules that those decision variables should satisfy. These include limitations or requirements, such as resource capacities or demand fulfillment.

Each of these components plays a crucial role in formulating a well-structured optimization model that accurately represents the problem and provides actionable solutions. The role of Sets will become more important as we get into more complicated optimization models.

Example 3.3: Bakery Production

Excel PuLP Gurobipy

A small bakery produces three products each day: cakes, cookies, and muffins. Each item requires a specific amount of ingredients and baking time, and each contributes to the bakery's profit. The bakery operates with daily limits on flour, sugar, and baking time due to ingredient supply and staffing constraints.

Each cake earns a profit of \$6.00, requires 500 grams of flour, 200 grams of sugar, and 60 minutes of baking time. Each cookie earns a profit of \$1.50, requires 100 grams of flour, 50 grams of sugar, and 10 minutes of baking time. Each muffin earns a profit of \$2.50, requires 200 grams of flour, 80 grams of sugar, and 20 minutes of baking time.

The bakery has at most 4,800 grams of flour, 2,060 grams of sugar, and 480 minutes of baking time available each day.

Determine how many cakes, cookies, and muffins the bakery should produce each day in order to maximize its total profit, while staying within the limits on flour, sugar, and baking time.

Linear Programming Model

$$\begin{aligned}
 &\text{Maximize} && 6x_1 + 1.5x_2 + 2.5x_3 \\
 &\text{subject to} && 500x_1 + 100x_2 + 200x_3 \leq 4800 \quad (\text{Flour}) \\
 &&& 200x_1 + 50x_2 + 80x_3 \leq 2060 \quad (\text{Sugar}) \\
 &&& 60x_1 + 10x_2 + 20x_3 \leq 480 \quad (\text{Baking time}) \\
 &&& x_1, x_2, x_3 \geq 0 \\
 &&& x_1, x_2, x_3 \in \mathbb{Z} \quad (\text{if integer production is required})
 \end{aligned}$$

Example 3.4: Production with welding robot

Excel PuLP Gurobipy

You have 21 units of transparent aluminum alloy (TAA), LazWeld1, a joining robot leased for 23 hours, and CrumCut1, a cutting robot leased for 17 hours of aluminum cutting. You also have production code for a bookcase, desk, and cabinet, along with commitments to buy any of these you can produce for \$18, \$16, and \$10 apiece, respectively. A bookcase requires 2 units of TAA, 3 hours of joining, and 1 hour of cutting, a desk requires 2 units of TAA, 2 hours of joining, and 2 hour of cutting, and a cabinet requires 1 unit of TAA, 2 hours of joining, and 1 hour of cutting.

Formulate an LP to maximize your revenue given your current resources.

Solution.

Sets:

- The types of objects = { bookcase, desk, cabinet }.

Parameters:

- Purchase cost of each object
- Units of TAA needed for each object
- Hours of joining needed for each object
- Hours of cutting needed for each object
- Hours of TAA, Joining, and Cutting available on robots

Decision variables:

x_i : number of units of product i to produce,
for all i =bookcase, desk, cabinet.

Objective and Constraints:

$$\begin{aligned}
 \max \quad & z = 18x_1 + 16x_2 + 10x_3 && \text{(profit)} \\
 \text{s.t.} \quad & 2x_1 + 2x_2 + 1x_3 \leq 21 && \text{(TAA)} \\
 & 3x_1 + 2x_2 + 2x_3 \leq 23 && \text{(LazWeld1)} \\
 & 1x_1 + 2x_2 + 1x_3 \leq 17 && \text{(CrumCut1)} \\
 & x_1, x_2, x_3 \geq 0.
 \end{aligned}$$



Example 3.5: The Diet Problem

Excel PuLP Gurobipy

In the future (as envisioned in a bad 70's science fiction film) all food is in tablet form, and there are four types, green, blue, yellow, and red. A balanced, futuristic diet requires, at least 20 units of Iron, 25 units of Vitamin B, 30 units of Vitamin C, and 15 units of Vitamin D. Formulate an LP that ensures a balanced diet at the minimum possible cost.

Tablet	Iron	B	C	D	Cost (\$)
Chem 1	6	6	7	4	1.25
Chem 2	4	5	4	9	1.05
Chem 3	5	2	5	6	0.85
Chem 4	3	6	3	2	0.65

Solution. Sets:

- Set of tablets {1,2,3,4}

Parameters:

- Iron in each tablet
- Vitamin B in each tablet
- Vitamin C in each tablet
- Vitamin D in each tablet
- Cost of each tablet

Decision variables:

x_i : number of tablet of type i to include in the diet, $\forall i \in \{1, 2, 3, 4\}$.

Objective and Constraints:

$$\begin{array}{ll}
 \text{Min} & z = 1.25x_1 + 1.05x_2 + 0.85x_3 + 0.65x_4 \\
 \text{s.t.} & 6x_1 + 4x_2 + 5x_3 + 3x_4 \geq 20 \\
 & 6x_1 + 5x_2 + 2x_3 + 6x_4 \geq 25 \\
 & 7x_1 + 4x_2 + 5x_3 + 3x_4 \geq 30 \\
 & 4x_1 + 9x_2 + 6x_3 + 2x_4 \geq 15 \\
 & x_1, x_2, x_3, x_4 \geq 0.
 \end{array}$$

**Example 3.6: The Next Diet Problem**

Excel PuLP Gurobipy

Progress is important, and our last problem had too many tablets, so we are going to produce a single, purple, 10 gram tablet for our futuristic diet requires, which are at least 20 units of Iron, 25 units of Vitamin B, 30 units of Vitamin C, and 15 units of Vitamin D, and 2000 calories. The tablet is made from blending 4 nutritious chemicals; the following table shows the units of our nutrients per, and cost of, grams of each chemical.

Formulate an LP that ensures a balanced diet at the minimum possible cost.

Tablet	Iron	B	C	D	Calories	Cost (\$)
Chem 1	6	6	7	4	1000	1.25
Chem 2	4	5	4	9	250	1.05
Chem 3	5	2	5	6	850	0.85
Chem 4	3	6	3	2	750	0.65

Solution.Sets:

- Set of chemicals $\{1, 2, 3, 4\}$

Parameters:

- Iron in each chemical
- Vitamin B in each chemical
- Vitamin C in each chemical
- Vitamin D in each chemical
- Cost of each chemical

Decision variables:

x_i : grams of chemical i to include in the purple tablet, $\forall i = 1, 2, 3, 4$.

Objective and Constraints:

$$\begin{array}{ll}
 \min & z = 1.25x_1 + 1.05x_2 + 0.85x_3 + 0.65x_4 \\
 s.t. & 6x_1 + 4x_2 + 5x_3 + 3x_4 \geq 20 \\
 & 6x_1 + 5x_2 + 2x_3 + 6x_4 \geq 25 \\
 & 7x_1 + 4x_2 + 5x_3 + 3x_4 \geq 30 \\
 & 4x_1 + 9x_2 + 6x_3 + 2x_4 \geq 15 \\
 & 1000x_1 + 250x_2 + 850x_3 + 750x_4 \geq 2000 \\
 & x_1 + x_2 + x_3 + x_4 = 10 \\
 & x_1, x_2, x_3, x_4 \geq 0.
 \end{array}$$

**Example 3.7: Work Scheduling Problem**

Excel PuLP Gurobipy

You are the manager of LP Burger. The following table shows the minimum number of employees required to staff the restaurant on each day of the week. Each employees must work for five consecutive days. Formulate an LP to find the minimum number of employees required to staff the restaurant.

Day of Week	Workers Required
1 = Monday	6
2 = Tuesday	4
3 = Wednesday	5
4 = Thursday	4
5 = Friday	3
6 = Saturday	7
7 = Sunday	7

Solution. This problem has multiple optimal solutions for which days workers begin working on, all of which result in 8 total workers hired.

Decision variables:

x_i : the number of workers that start 5 consecutive days of work on day i , $i = 1, \dots, 7$

Objective and Constraints:

$$\begin{array}{ll}
 \text{Min} & z = x_1 + x_2 + x_3 + x_4 + x_5 + x_6 + x_7 \\
 \text{s.t.} & x_1 + x_4 + x_5 + x_6 + x_7 \geq 6 \\
 & x_2 + x_5 + x_6 + x_7 + x_1 \geq 4 \\
 & x_3 + x_6 + x_7 + x_1 + x_2 \geq 5 \\
 & x_4 + x_7 + x_1 + x_2 + x_3 \geq 4 \\
 & x_5 + x_1 + x_2 + x_3 + x_4 \geq 3 \\
 & x_6 + x_2 + x_3 + x_4 + x_5 \geq 7 \\
 & x_7 + x_3 + x_4 + x_5 + x_6 \geq 7 \\
 & x_1, x_2, x_3, x_4, x_5, x_6, x_7 \geq 0.
 \end{array}$$

One solution is as follows:

LP Solution	IP Solution
$z_{LP} = 7.333$	$z_I = 8.0$
$x_1 = 0$	$x_1 = 0$
$x_2 = 0.333$	$x_2 = 0$
$x_3 = 1$	$x_3 = 0$
$x_4 = 2.333$	$x_4 = 3$
$x_5 = 0$	$x_5 = 0$
$x_6 = 3.333$	$x_6 = 4$
$x_7 = 0.333$	$x_7 = 1$



Example 3.8: LP Burger - extended

Excel PuLP Gurobipy

LP Burger has changed its policy, and allows, at most, two part time workers, who work for two consecutive days in a week. Formulate this problem.

Solution. Decision variables:

x_i : the number of workers that start 5 consecutive days of work on day i , $i = 1, \dots, 7$

y_i : the number of workers that start 2 consecutive days of work on day i , $i = 1, \dots, 7$.

Objective and Constraints:

Min

$$z = 5(x_1 + x_2 + x_3 + x_4 + x_5 + x_6 + x_7) \\ + 2(y_1 + y_2 + y_3 + y_4 + y_5 + y_6 + y_7)$$

s.t.

$$x_1 + x_4 + x_5 + x_6 + x_7 + y_1 + y_7 \geq 6$$

$$x_2 + x_5 + x_6 + x_7 + x_1 + y_2 + y_1 \geq 4$$

$$x_3 + x_6 + x_7 + x_1 + x_2 + y_3 + y_2 \geq 5$$

$$x_4 + x_7 + x_1 + x_2 + x_3 + y_4 + y_3 \geq 4$$

$$x_5 + x_1 + x_2 + x_3 + x_4 + y_5 + y_4 \geq 3$$

$$x_6 + x_2 + x_3 + x_4 + x_5 + y_6 + y_5 \geq 7$$

$$x_7 + x_3 + x_4 + x_5 + x_6 + y_7 + y_6 \geq 7$$

$$y_1 + y_2 + y_3 + y_4 + y_5 + y_6 + y_7 \leq 2$$

$$x_i \geq 0, y_i \geq 0, \forall i = 1, \dots, 7.$$



3.3.1. Knapsack Problem

We consider now a simple example where the variables can only take the values of 0 or 1. We call these *binary variables*.

Example 3.9: Camping Trip

Excel PuLP Gurobipy

Imagine you are preparing for a week-long camping trip to the mountains. You have a backpack with a weight capacity of 20 kilograms. Your goal is to pack the most valuable items to ensure a comfortable and safe trip without exceeding the backpack's weight limit. Each item has a weight and a value associated with it, representing its importance and utility for the trip. The table below lists the potential items you can take, their weights in kilograms, and their values on a scale from 1 to 10 (with 10 being the most valuable).

Formulate an Integer Program to find the optimal items you should bring on your trip with you.

Item	Index	Weight (kg)	Value
Tent	A	4	10
Stove	B	2	9
Sleeping Bag	C	3	8
First Aid Kit	D	1	7
Food Supplies	E	5	9
Water Filter	F	1	6
Clothes	G	4	5
Map and Compass	H	0.5	7

Solution. Decision variables:

x_i : whether to include item i in the backpack, where $i \in \{A, B, C, D, E, F, G, H\}$, $x_i = 1$ if item i is included, and $x_i = 0$ otherwise.

Objective and Constraints:

$$\begin{aligned}
 \text{Max } z &= 10x_A + 9x_B + 8x_C + 7x_D + 9x_E + 6x_F + 5x_G + 7x_H \\
 \text{s.t. } &4x_A + 2x_B + 3x_C + 1x_D + 5x_E + 1x_F + 4x_G + 0.5x_H \leq 20 \\
 &x_A, x_B, x_C, x_D, x_E, x_F, x_G, x_H \in \{0, 1\}.
 \end{aligned}$$



3.3.2. Capital Investment

We next see a similar application of binary variables.

Example 3.10: Capital Allocation Problem

Excel PuLP Gurobipy

You are a financial planner for an investment firm. The firm has \$100,000 to invest in a portfolio of projects. Each project has a projected return and requires an initial investment. The goal is to maximize the total return of the portfolio while not exceeding the available capital. The following table lists potential projects, their required investments, and their projected returns.

Formulate an LP to maximize the total return of the portfolio while not exceeding the available investment capital.

Project	Investment Required (\$)	Projected Return (\$)
A	20,000	30,000
B	30,000	40,000
C	50,000	75,000
D	10,000	15,000
E	40,000	60,000

Solution.

Sets:

- Projects: $P = \{A, B, C, D, E\}$

Parameters:

- inv_i : investment required for project i
- ret_i : projected return from project i
- $B = 100,000$: total available capital

Decision Variables:

$x_i \in \{0, 1\}$: 1 if project i is selected, 0 otherwise

Objective and Constraints:

$$\begin{aligned}
 \max \quad & z = 30,000x_A + 40,000x_B + 75,000x_C + 15,000x_D + 60,000x_E \\
 \text{s.t.} \quad & 20,000x_A + 30,000x_B + 50,000x_C + 10,000x_D + 40,000x_E \leq 100,000 \\
 & x_A, x_B, x_C, x_D, x_E \in \{0, 1\}
 \end{aligned}$$

Compact Form:

$$\begin{aligned}
& \max \quad \sum_{i \in P} \text{ret}_i \cdot x_i \\
& \text{s.t.} \quad \sum_{i \in P} \text{inv}_i \cdot x_i \leq B \\
& \quad x_i \in \{0, 1\} \quad \forall i \in P
\end{aligned}$$



3.3.3. Transportation problem

Example 3.11: Solar Panel Shipping

Excel PuLP Gurobipy

SolTranz Inc. manufactures high-efficiency solar panels at three factories located in Reno (R), Boulder (B), and Tucson (T). The maximum production at each facility is 4,500, 5,500, and 7,000 panels, respectively. These panels must be shipped to five warehouses (labeled 1 through 5), each with a fixed demand.

The cost of producing a solar panel depends on the factory, and the shipping costs to each warehouse are also known. However, no more than 2,000 panels can be shipped from a factory to a single warehouse. The table below summarizes the data.

Formulate a linear program that minimizes the total cost of production and shipping to satisfy all warehouse demands.

Factory	Prod. Cost (\$)	WH1	WH2	WH3	WH4	WH5
Reno (R)	150	60	70	75	65	50
Boulder (B)	140	80	55	65	70	60
Tucson (T)	130	85	60	55	50	65

Shipping cost from each factory to each warehouse (in \$ per panel).

Warehouse Demands:

WH1: 3000 WH2: 3500 WH3: 2500 WH4: 4000 WH5: 3000

Solution.

Sets:

- Factories: $F = \{R, B, T\}$
- Warehouses: $W = \{1, 2, 3, 4, 5\}$

Parameters:

- C_f : production cost per panel at factory f
- S_f : supply capacity of factory f
- D_w : demand at warehouse w
- $T_{f,w}$: shipping cost per panel from factory f to warehouse w
- $M = 2000$: max panels shipped from any factory to any warehouse

Decision Variables:

$x_{f,w}$: number of solar panels shipped from factory f to warehouse w

Objective and Constraints:

$$\begin{aligned}
 \min \quad & \sum_{f \in F} \sum_{w \in W} (C_f + T_{f,w}) \cdot x_{f,w} \\
 \text{s.t.} \quad & \sum_{w \in W} x_{R,w} \leq 4500 && \text{(Reno capacity)} \\
 & \sum_{w \in W} x_{B,w} \leq 5500 && \text{(Boulder capacity)} \\
 & \sum_{w \in W} x_{T,w} \leq 7000 && \text{(Tucson capacity)} \\
 & \sum_{f \in F} x_{f,1} = 3000 && \text{(WH1 demand)} \\
 & \sum_{f \in F} x_{f,2} = 3500 && \text{(WH2 demand)} \\
 & \sum_{f \in F} x_{f,3} = 2500 && \text{(WH3 demand)} \\
 & \sum_{f \in F} x_{f,4} = 4000 && \text{(WH4 demand)} \\
 & \sum_{f \in F} x_{f,5} = 3000 && \text{(WH5 demand)} \\
 & 0 \leq x_{f,w} \leq 2000 && \forall f \in F, \forall w \in W
 \end{aligned}$$



3.3.4. Assignment Problem

Consider the assignment of 3 employees to 3 tasks, where each employee has a cost to do each task. How to figure out the best assignment to the tasks?

Example 3.12: Hiring for tasks

Excel PuLP Gurobipy

In this assignment problem, we need to hire three people (Person 1, Person 2, Person 3) to three tasks (Task 1, Task 2, Task 3). In the table below, we list the cost of hiring each person for each task, in dollars. Since each person has a different cost for each task, we must make an assignment to minimize

	Cost	Task 1	Task 2	Task 3
our total cost.	Person 1	40	47	80
	Person 2	72	36	58
	Person 3	24	61	71

Given the specific costs of assigning three people to three tasks, we can write out the mathematical model explicitly using the given numbers.

Solution.

Sets: Set of tasks and set of persons

Parameters: Cost c_{ij} for person i to complete task j .

Variables: Let $x_{ij} = 1$ if person i is assigned to task j and 0 otherwise.

Objective Function:

Minimize the total cost of assignments:

$$z = 40x_{11} + 47x_{12} + 80x_{13} + 72x_{21} + 36x_{22} + 58x_{23} + 24x_{31} + 61x_{32} + 71x_{33} \quad (3.1)$$

Subject to Constraints:

Each person is assigned to exactly one task:

$$x_{11} + x_{12} + x_{13} = 1 \quad (3.2)$$

$$x_{21} + x_{22} + x_{23} = 1 \quad (3.3)$$

$$x_{31} + x_{32} + x_{33} = 1 \quad (3.4)$$

Each task is assigned to exactly one person:

$$x_{11} + x_{21} + x_{31} = 1 \quad (3.5)$$

$$x_{12} + x_{22} + x_{32} = 1 \quad (3.6)$$

$$x_{13} + x_{23} + x_{33} = 1 \quad (3.7)$$

Binary constraints on the variables:

$$x_{ij} \in \{0, 1\} \quad \forall i \in \{1, 2, 3\}, \forall j \in \{1, 2, 3\} \quad (3.8)$$



3.4 Exercises

Exercise 3.13 Which ones are linear functions?

1. $x^2 + 3y + 2z$
2. $\sqrt{x_1 x_2 x_3}$
3. $\sum_{i=1}^n a_i 2^{x_i}$, where $a_1, a_2, \dots, a_n$ are constant
4. $\sum_{i=1}^n a_i x_i$, where $a_1, a_2, \dots, a_n$ are constant

Exercise 3.14 Which one is a linear function?

1. $2x_1 + x_2 x_3 - 7x_3$
2. $\sqrt{2}x_1 + x_2 + 3^{1.6}x_3$

Exercise 3.15 Which ones are linear inequalities? Can any be made into linear inequalities?

1. $\frac{x_1}{x_1 + x_2 + x_3} \geq 0.5$
2. $\sin x_1 + x_2 + x_3 = 0.5$
3. $\sum_{i=1}^n a_i 2^{x_i} \geq b$, where $a_1, a_2, \dots, a_n, b$ are constant
4. $\sum_{i=1}^n a_i x_i = b$, where $a_1, a_2, \dots, a_n, b$ are constant
5. $\sum_{i=1}^n a_i x_i \geq b$, where $a_1, a_2, \dots, a_n, b$ are constant

Exercise 3.16 An appliance store sells three types of ovens: small, medium, and large. The small, medium, and large ovens require, respectively, 3, 5, and 6 cubic feet of storage space; at most 100 cubic feet of storage is available. Each oven takes 1 hour of sales time; there is a maximum of 200 hours of sales labor time available. The small, medium, and large ovens require, respectively, 1, 1, and 2 hours of installation time; at most 280 hours of installer labor are available monthly.

If the profit (in dollars) from small, medium, and large ovens is 50, 100, and 150 respectively, how many of each type of oven should be sold to maximize profit, and what is the maximum profit?

Exercise 3.17 A factory manufactures three products, A, B, and C. Each product requires the use of two machines (Machine I and Machine II). The total hours available per month on Machine I and Machine II

are 180 and 300, respectively. The table below shows the time requirements (in hours per unit) and profit per unit for each product.

	A	B	C
Machine I	1	2	2
Machine II	2	2	4
Profit (dollars)	20	30	40

How many units of each product should be manufactured to maximize profit, and what is the maximum profit?

Exercise 3.18 A company produces three products, A, B, and C, at two factories, Factory I and Factory II. Daily production of each factory for each product is listed below:

	Factory I	Factory II
Product A	10	20
Product B	20	20
Product C	20	10

The company must produce at least 1000 units of product A, 1600 units of product B, and 700 units of product C. If the cost of operating Factory I is \$4000 per day and the cost of operating Factory II is \$5000 per day, how many days should each factory operate to complete the order at a minimum cost, and what is the minimum cost?

Exercise 3.19 Professor Wright gives three types of quizzes (objective, recall, and recall-plus) and decides to give at least 20 quizzes next quarter. Each type of quiz requires the students to spend extra preparation time:

Objective: 10 minutes per quiz,

Recall: 30 minutes per quiz,

Recall-plus: 60 minutes per quiz.

He wants students to spend at least 12 hours (720 minutes) in total preparation for these quizzes (beyond normal study time).

Average quiz scores are:

Objective: 5, Recall: 6, Recall-plus: 7.

He wants the total of all quiz scores to be at least 130 points.

It takes the professor:

1 minute to grade each objective quiz,

2 minutes to grade each recall quiz,

3 minutes to grade each recall-plus quiz.

How many of each type should he give in order to minimize his own grading time?

These exercises were shared under a CC BY 4.0 license and was authored, remixed, and/or curated by Rupinder Sekhon and Roberta Bloom via source content that was edited to the style and standards of the LibreTexts platform.

3.5 Contributors



Champions of Access to Knowledge



OPEN TEXT

All digital forms of access to our high-quality open texts are entirely FREE! All content is reviewed for excellence and is wholly adaptable; custom editions are produced by Lyryx for those adopting Lyryx assessment. Access to the original source files is also open to anyone!



ONLINE ASSESSMENT

We have been developing superior online formative assessment for more than 15 years. Our questions are continuously adapted with the content and reviewed for quality and sound pedagogy. To enhance learning, students receive immediate personalized feedback. Student grade reports and performance statistics are also provided.



SUPPORT

Access to our in-house support team is available 7 days/week to provide prompt resolution to both student and instructor inquiries. In addition, we work one-on-one with instructors to provide a comprehensive system, customized for their course. This can include adapting the text, managing multiple sections, and more!



INSTRUCTOR SUPPLEMENTS

Additional instructor resources are also freely accessible. Product dependent, these supplements include: full sets of adaptable slides and lecture notes, solutions manuals, and multiple choice question banks with an exam building tool.

Contact Lyryx Today!

info@lyryx.com

¹This book was not produced by Lyryx, but this book has made substantial use of their open source material. We leave this page in here as a tribute to Lyryx for sharing their content.

lyryx
advancing learning
A First Course in Linear Algebra
an Open Text

BE A CHAMPION OF OER!

Contribute suggestions for improvements, new content, or errata:

A new topic

A new example

An interesting new question

A new or better proof to an existing theorem

Any other suggestions to improve the material

Contact Lyryx at info@lyryx.com with your ideas.

CONTRIBUTIONS

Ilijas Farah, York University

Ken Kuttler, Brigham Young University

Lyryx Learning Team

Foundations of Applied Mathematics

<https://github.com/Foundations-of-Applied-Mathematics>

CONTRIBUTIONS

List of Contributors

E. Evans

Brigham Young University

R. Evans

Brigham Young University

J. Grout

Drake University

J. Humpherys

Brigham Young University

T. Jarvis

Brigham Young University

J. Whitehead

Brigham Young University

J. Adams

Brigham Young University

J. Bejarano

Brigham Young University

Z. Boyd

Brigham Young University

M. Brown

Brigham Young University

A. Carr

Brigham Young University

C. Carter

Brigham Young University

T. Christensen

Brigham Young University

M. Cook

Brigham Young University

R. Dorff

Brigham Young University

B. Ehlert

Brigham Young University

M. Fabiano

Brigham Young University

K. Finlinson

Brigham Young University

J. Fisher

Brigham Young University

R. Flores

Brigham Young University

R. Fowers

Brigham Young University

A. Frandsen

Brigham Young University

R. Fuhrman

Brigham Young University

S. Giddens

Brigham Young University

C. Gigena

Brigham Young University

M. Graham

Brigham Young University

F. Glines

Brigham Young University

C. Glover

Brigham Young University

M. Goodwin

Brigham Young University

R. Grout

Brigham Young University

D. Grundvig

Brigham Young University

E. Hannesson

Brigham Young University

J. Hendricks

Brigham Young University

A. Henriksen

Brigham Young University

I. Henriksen

Brigham Young University

C. Hettinger

Brigham Young University

S. Horst
Brigham Young University
K. Jacobson
Brigham Young University
J. Leete
Brigham Young University
J. Lytle
Brigham Young University
R. McMurray
Brigham Young University
S. McQuarrie
Brigham Young University
D. Miller
Brigham Young University
J. Morrise
Brigham Young University
M. Morrise
Brigham Young University
A. Morrow
Brigham Young University
R. Murray
Brigham Young University
J. Nelson
Brigham Young University
E. Parkinson
Brigham Young University
M. Probst
Brigham Young University
M. Proudfoot
Brigham Young University
D. Reber
Brigham Young University

H. Ringer
Brigham Young University
C. Robertson
Brigham Young University
M. Russell
Brigham Young University
R. Sandberg
Brigham Young University
C. Sawyer
Brigham Young University
M. Stauffer
Brigham Young University
J. Stewart
Brigham Young University
S. Suggs
Brigham Young University
A. Tate
Brigham Young University
T. Thompson
Brigham Young University
M. Victors
Brigham Young University
J. Webb
Brigham Young University
R. Webb
Brigham Young University
J. West
Brigham Young University
A. Zaitzeff
Brigham Young University

This project is funded in part by the National Science Foundation, grant no. TUES Phase II DUE-1323785.